摘　要

最近几年，3D 打印技术快速发展，逐步走进人们的日常生活，其中，基于 FDM 技术的 3D 打印由于成本低廉，操作简单，易于制作，系统安全，个性定制等等优点，被广泛应用于教育、医疗、军事、航空航天等多个领域。得益于高速打印技术的进一步开发和愈加完善的 3D 打印工厂，打印产品的生产速度进一步提高，但更快的生产也暴露出了 FDM 3D 打印在生产成型过程中容易出现各种缺陷的问题。

本文针对解决 FDM 3D 打印过程出现的缺陷，提出了利用机器视觉结合深度学习神经网络的检测方法。通过对打印缺陷进行分析总结，收集打印缺陷的图像数据，经过处理后交给建立好的模型进行训练，不断调整参数，提高模型性能。最后在模型的基础上，创新设计了显示缺陷数据的 GUI 界面，方便使用者判断缺陷的各项数据，还可直接保存缺陷数据，使用简便快捷，可以极大提高缺陷检测的效率和准确率。若将其投入实际应用进一步普及，还能提高检测准确率和检测范围。

关键词：FDM 3D 打印；机器视觉；缺陷识别

Abstract

In recent years, 3D printing technology has developed rapidly and gradually entered people's daily lives. Among them, FDM-based 3D printing has been widely applied in education, medical, military, aerospace and other fields due to its advantages such as low cost, simple operation, easy manufacturing, system safety, and personalized customization. Thanks to the further development of high-speed printing technology and increasingly improved 3D printing factories, the production speed of printed products has been further increased. However, faster production has also exposed the problem that FDM 3D printing is prone to various defects during the forming process.

This thesis proposes a detection method combining machine vision and deep learning neural networks to solve the defects in the FDM 3D printing process. By analyzing and summarizing printing defects, collecting image data of defects, processing the data, and feeding it into an established model for training,the parameters are continuously adjusted to improve the model's performance. Finally, based on the model, a GUI interface for displaying defect data is innovatively designed, which helps users judge various defect parameters and directly save the data, ensuring simple and fast operation. This can greatly improve the efficiency and accuracy of defect detection. If put into practical application and further popularized, it can also enhance the detection accuracy and expand the detection scope.

Key Words: FDM 3D Printing; Machine Vision; Defect Recognition

目　录

第 1 章 绪论 1
1.1 研究背景、目的及意义 1
1.2 国内外研究现状 1
1.2.1 FDM 3D 打印缺陷成因分析 1
1.2.2 关于 FDM 的 3D 打印缺陷检测技术的研究 4
1.2.3 传统方法和深度学习方法分析 5
1.2.4 深度学习算法比较分析 6
1.3 研究内容与方法 8
1.3.1 研究内容 8
1.3.2 研究方法 9
第 2 章 实时缺陷检测平台的搭建 10
2.1 搭建图像采集系统 10
2.2 收集整理缺陷数据 10
2.2.1 采集缺陷图片建立数据库 10
2.2.2 数据预处理与增强 12
2.3 基于 YOLOv10 搭建模型 13
2.3.1 Yolov10 的网络架构 13
2.3.2 检测原理 14
第 3 章 检测模型训练 15
3.1 网络训练 15
3.1.1 激活函数的选择 15
3.1.2 损失函数的选择 15
3.2 调整检测模型 16
3.2.1 调整训练参数 16
3.2.2 性能对比 17
3.3 检测功能检验 19
第 4 章 设计人机交互界面 23
4.1 界面设计 23
4.2 界面功能检验 24
第 5 章 结论与展望 27
参考文献 28
致　谢 30
附录 A：程序清单 32
附录 B：外文翻译资料 41

第1章 绪论

1.1 研究背景、目的及意义

FDM 3D 打印技术正在不断突破过去的传统应用，从最开始的打印机原型、应用设想，现如今已经广泛应用于各个领域，大到教育、医疗、军事、航空航天等大型领域，小到模型制作、零件打印、玩具打印甚至 3D 摄影打印。有人把他当作数字化教具，在扩充中小学生的课外知识的同时，也为人们带来欢声笑语；有人用他拯救生命，打印心脏血管、人体骨骼、人造耳蜗、牙齿，个性化定制人造器官，为人们带去希望；也有人用他打印一体化的航空零件，助力人类航空航天事业更高更远[1][2]；还有人用他打印出脑海中的设计，有建筑、有衣装、有玩具也有饰品[3]。是的，FDM 3D 打印为各个行业带来新的可能、新的机遇，更加快捷、更加灵活的方案为行业的发展注入了新的活力，近些年，高速打印技术和 3D 打印工厂的诞生和发展，使得 3D 打印产品得以更快的被生产出来，更多的人得以享受到这一新技术。

随着高速打印技术和 FDM 3D 打印工厂的发展，在保证打印的质量的同时也要求追求打印速度，这两者往往不能兼顾。FDM 3D 打印机的性能和配置直接影响打印效果和质量，在打印前需要调整和优化打印机的层高、喷嘴直径、温度控制等参数，在打印过程中需要进行多次调试和控制，以确保打印过程的稳定和物体的准确性[4]。然而，当使用场景要求根据大量的实际情况快速准确的调整配置和参数，人类操作员往往无法满足这种要求。虽然有默认设置，但是直接依靠默认设置打印往往得不到满意的的打印成果。由于材料、打印设备、打印参数等多重因素的影响，3D 打印部件常常会出现各种缺陷，这对部件的性能和可靠性产生了重大影响，导致打印成品质量无法保证，给操作者带来困难的同时也极大地阻碍了 3D 打印技术的应用与发展[4]。因此，如何快速分析缺陷、调整打印参数、提高打印效率的同时控制 3D 打印的成本是值得深入研究的一项课题。

本项目提出一种基于机器视觉和深度学习的 FDM 3D 打印缺陷实时检测方法，采用 CCD 相机进行图像采集，在图像处理后形成数据库，经过神经网络模型的训练和验证，即时反应何时何处发生了打印缺陷，最终通过人机交互界面给出解决方案，从而实现 FDM 3D 打印过程中的缺陷识别检测。

1.2 国内外研究现状

1.2.1 FDM 3D 打印缺陷成因分析

从目前的文献中可以发现，FDM 缺陷主要源于材料特性、工艺参数和环境条件。比如，材料冷却不均导致层间结合力弱；挤出流量不稳定或路径规划缺陷引发内部空

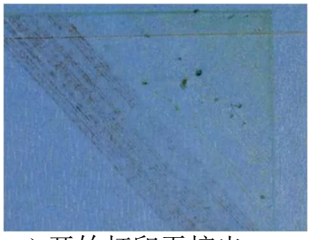
洞；热塑性材料收缩差异引起基底剥离；层间台阶效应和支撑结构残留导致表面质量下降等等。想要通过机器视觉的方法对打印过程和打印件进行检测并识别存在的故障与缺陷，并针对缺陷给出相应的解决办法，就需要了解 FDM 3D 打印不同缺陷形成的机理。如表 1-1 和

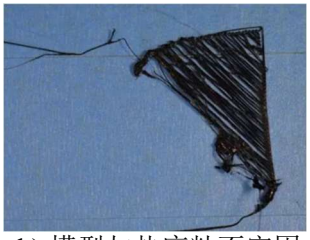
图 1-1 总结了 FDM 3D 打印常见缺陷表现及其成因与解决思路。

表 1-1　FDM 3D 打印常见缺陷的成因与解决思路

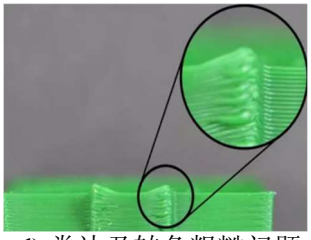
缺陷类型	主要成因	解决思路
层间粘附不良	喷嘴温度过低；平台温度低；打印速度快；冷却风扇强度过高。	提高喷嘴/平台温度；降低打印速度；减少冷却风扇功率。
翘边（边缘卷曲）	材料冷却收缩不均匀；平台附着力不足；环境温度或气流干扰。	提高平台温度；使用胶水/PEI 板；关闭通风，保持环境稳定。
拉丝/渗漏	回抽参数不当；喷嘴温度过高；非打印移动时未完全停止挤出。	优化回抽距离和速度；降低喷嘴温度；启用“Z-hop”功能。
填充结构不全	填充密度过低；喷嘴堵塞或挤出倍率错误）；打印速度过快。	增加填充密度；校准挤出倍率；降低填充速度。
第一层问题	平台未调平；首层速度过快；挤出不足（首层挤出倍率低）。	重新调平平台；降低首层速度；增加首层挤出倍率。
挤出不足/过度	挤出机齿轮磨损；耗材直径不均；喷嘴堵塞或喉管散热差。	校准挤出齿轮；使用高质量耗材；清理喷嘴，检查散热风扇。
层错位	电机失步（速度/加速度过高）；皮带松动；外部碰撞或振动。	降低打印速度/加速度；张紧皮带；确保打印机放置稳定。
顶部表面凹陷	顶部实心层数不足；填充密度低；冷却不足导致收缩。	增加顶部层数；提高填充密度；优化冷却强度。
材料相关缺陷	耗材受潮；脆性断裂；颜色不均（混合批次或残留旧料）。	烘干耗材；调整打印温度；清理喷嘴或避免混用耗材。

研究学者从不同的角度对 FDM 3D 打印缺陷形成因素进行了研究，通过归纳总结为 3 个成因：（1）打印设备引起的打印缺陷；（2）切片软件参数设置不当；（3）未考虑环境因素。有些缺陷是设备调节可避免的，有些可通过工艺参数调节可避免的，而有些是熔丝工艺本身造成的。

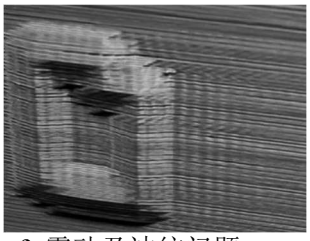

a) 开始打印无挤出

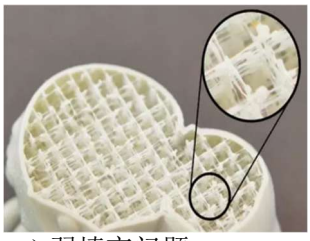

b) 模型与热床粘不牢固

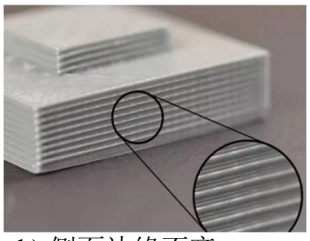
c) 层分离问题

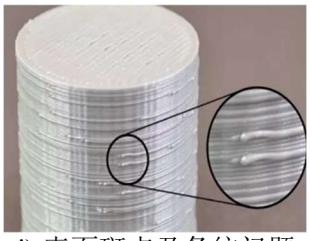

d) 卷边及转角粗糙问题

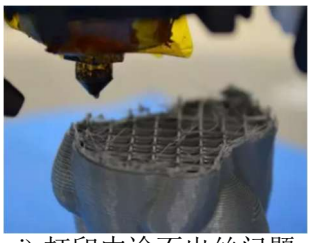
e) 偏移问题

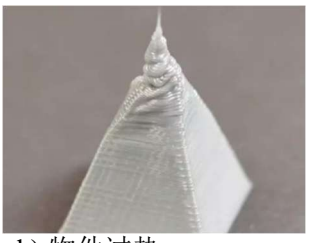

f) 震动及波纹问题

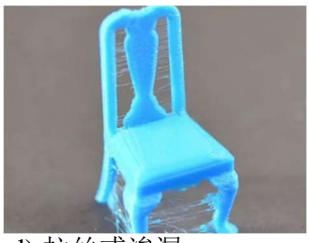

g) 弱填充问题

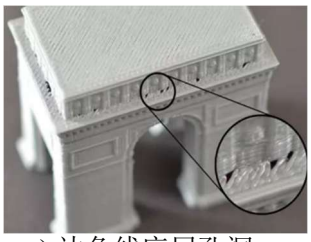

h) 侧面边缘不齐


i) 表面斑点及条纹问题

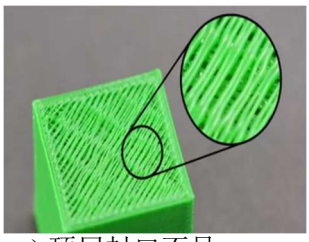

j) 打印中途不出丝问题

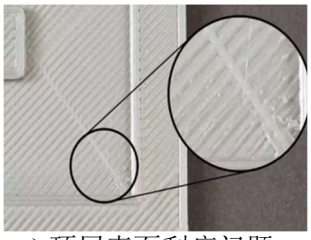

k) 物件过热

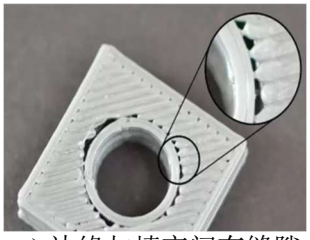

l) 拉丝或渗漏


m) 边角线底层孔洞

n) 薄墙缝隙问题


o) 顶层封口不足


p) 顶层表面刮痕问题


q) 边缘与填充间有缝隙

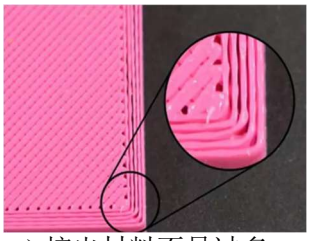

r) 挤出材料不足过多

图 1-1 FDM 3D 打印缺陷具体表现

1.2.2 关于 FDM 的 3D 打印缺陷检测技术的研究

在各种 3D 打印技术中 FDM（熔融沉积成型）技术通过逐层堆积熔融打印材料的方法，达到了降低成本和操作难度的目的，使更多人易于上手打印操作，而且可以选用多种耗材，使得打印多样化，他已经成为目前应用最广的 3D 打印技术。然而，这种层层堆积材料的工艺决定了其打印产品容易出现各种各样的缺陷，比如孔隙、层间黏附不良、翘曲变形等缺陷，直接影响制件的力学性能和尺寸精度[6]。因此，研究者们在这些年从多个方向寻找降低打印缺陷率，提高打印产品质量的方法，针对 FDM 缺陷检测技术的研究因此得到了重视。

打印层间错位引起的纹理畸变让 Mohamed 等人十分苦恼，针对这个问题，他的团队想到了用灰度共生矩阵量化缺陷特征，也就是通过机器视觉获取图像然后用数字图像处理技术（灰度共生矩阵）进行处理，他们根据这个原理开发的缺陷检测模型取得了 89%的检测精度[6]。

在多轴 FDM 打印机上，张海光等人提出了一种基于机器视觉与卷积神经网络（CNN）相结合的 FDM 打印检测方法，在研究过程中他们只选取了一种层间剥离缺陷作为检测对象，使用 CCD 相机采集打印件的状态图像构建数据集，然后构建 CNN 网络对收集到的图像进行缺陷识别与分类，随后通过训练与验证一次次改进网络参数，最终得到了性能优异的 CNN 网络[7]，本文中我的研究思路也受到了他们的启发。

张月桐等人提出了一种基于红外成像的 FDM 打印轨迹测量及质量评价方法，采用红外相机对 FDM 打印机的热加工过程进行连续测量，通过建立打印机系统坐标系与相机世界坐标系的位姿解算模型，以及动态帧间红外特征匹配的全局相机运动位姿解算模型，实现了对末端喷嘴运行位姿的实时测量，得到了精确的打印轨迹信息，并在图像域对打印过程中的典型缺陷进行了质量评价[8]。

李飞等人通过三维视觉的方法获取到打印零件的表面点云数据，然后结合深度学习技术，最终实现了对 3D 打印零件表面的缺陷检测[9]。并且在此基础上进一步研究，采集模型上表面的正投影，据此对打印精度进行逐层分析；收集模型顶层的轮廓信息以及工作设备的轮廓信息，可以对喷头运行状态进行实时检测；以及用主动扫描的方式对打印模型进行三维重建等方法，这些方法的应用可以很好地检测到设备异常运行造成的缺陷，但也存在不足，就是对材料热成型产生的缺陷检测效果不好。

陈旭和李筠提出了一种基于三维激光成像技术的 3D 打印表面成型缺陷自动检测方法，利用三维激光成像系统的扫描仪获取 3D 打印表面成型材料的三维激光点云数据，经过粗配准和精配准后，利用四象限临近点搜索算法获取 3D 打印表面成型缺陷点的三维坐标值，有效检测了孔洞、打印错位等不同类型缺陷[10]。

王红梅结合 AI（人工智能）和额外特征卷积层设计出一种全新的缺陷检测器

（DDAI），在收集了 500 张图片完成训练后她的测试结果表明 DDAI 能够低成本检测出 3D 打印目标的缺陷，文中图像采集系统为我本文设计的 FDM 打印缺陷检测系统提供了灵感[11]。

弗吉尼亚大学的 Zackary 和 Luke 对产品进行原位缺陷检测研究，在排除了红外成像、X 射线的检测方法后借助机器学习算法构建出基于卷积神经网络的检测模型，对匙孔不稳定这一缺陷实现了接近 100%的亚毫秒级分辨能力[12]，对本研究的算法构建有重要参考意义。

王小斌和高爽等人解决了集成电路板（IC）的表面缺陷极度依赖人工检测员检测的问题，他们先是在打印过程中安装传感器收集检测信息，接着利用 ResNet50 模型和 Vision Transformer （ViT） 模型的改进体提取特征并对这些综合数据进行学习训练，在测试集上的实验数据可达到 98.6%的高准确率 [14]。

孙伟俊设计的机器人 3D 打印视觉检测系统，摄像头可以在机械臂的控制下沿设定的轨迹运动，可以采集 3D 打印件的侧表面缺陷特征[15]，确实达到了多角度观测 3D 打印件的目的尽管如此这个视觉监测系统也存在不能检测相层叠缺陷的问题，本研究的图像提取系统也参考了他的方法。

1.2.3 传统方法和深度学习方法分析

传统的缺陷检测方法，需要消耗很多的人力物力，要求检测人员需要有极高的专业素养的检测经验，需要熟悉打印件的缺陷特征、轮廓、纹理等等，即使引入机器助力，这也需要人工对缺陷特征进行标注，需要对特征进行定义。然后才能运用算法提取关键的缺陷信息进行训练，最终用模型完成识别任务

和传统的检测方法相比较，深度学习的方法可以自动化学习缺陷特征。其中最具代表性的卷积神经网络具有强大的学习能力，利用多层的审计网络架构逐层堆叠，可以达到自主提取基础元素和复杂特征的效果，包括打印件的线条、颜色、整体的形状等等。深度学习的方式摆脱了对特征标注的依赖，虽然还需要人工的参与，但是人的工作量已经大大减轻，深度学习用大量的数据优化了提取特征和分类特征的过程，这大大提高了模型的识别能力。

深度学习与机器学习是包含与被包含的关系，传统的机器学习追求更多的数据量来改进算法，深度学习作为机器学习的子集，其实是在数据和模型之间增加了一层神经网络用于处理数据，对于传统机器学习难以处理的复杂表面特征，深度学习算法可以模仿人的思考和活动帮助解决复杂的识别难题，使得他在缺陷检测中具有实时性强和适应性高的优势 [15]。

从特征提取和复杂任务的处理，深度学习确实具有很大优势，但传统的机器视觉检测方法在实时检测的应用场景里也有不可替代的优势。在实际的应用中，将传统机器视觉方法和深度学习相结合，发挥开两者的优点，先利用传统算法定位缺陷，然后

利用深度学习算法进行分类学习，取长补短，两者协同无疑是未来缺陷实时检测的主流。两者对比如表 1-2。

表 1-2　传统方法与深度学习方法对比

维度	机器视觉（传统方法）	深度学习	核心关系
核心技术	手工标注特征（SVM、卡尔曼滤波）	深度神经网络、端到端学习	深度学习实现了自动化特征提取
特征提取方式	依赖专家经验，提取局部或全局特征	通过多层网络自动学习从低级到高级的层次化特征	深度学习显著提升特征表达能力，减少人为干预
数据需求	小样本即可工作，依赖领域知识优化	需大量标注数据，数据量不足时易过拟合	传统方法适合小数据场景，深度学习依赖大数据驱动
计算资源	算力要求低，适合嵌入式设备	需 GPU 等高算力支持，模型训练成本高	传统方法更轻量，深度学习需硬件支撑
应用场景	工业质检、简单分类、实时性要求高的任务	复杂识别、自然场景理解	传统方法在结构化场景高效，深度学习解决复杂非结构化问题
可解释性	逻辑透明，决策过程可追溯	黑箱模型，决策过程难以解释	传统方法更适用于安全敏感领域
典型算法	Canny 边缘检测、Harris 角点检测	ResNet、YOLO、U-Net、Transformer	深度学习逐渐成为主流，但传统算法仍用于
发展趋势	与深度学习融合，作为预处理或补充逻辑	轻量化、自监督学习、多模态融合	两者互补：深度学习主导感知，传统方法辅助优化效率与可靠性
局限	复杂场景泛化能力差，依赖人工调参	数据与算力依赖高，可解释性差	结合两者优势可突破单一技术瓶颈

从上面的列表分析对比不难看出，深度学习更像是一种更强大的技术手段，而机器视觉则是其得以应用的领域。随着技术的发展，深度学习可以推动机器视觉缺陷检测更加智能更加高效、更加智能，与此同时还降低了检测成本，即使需要更多的图像用于建立数据库训练，这也是可以接受的，当应用于 FDM 3D 打印缺陷检测上时，节省的人力成本远高于收集数据的成本，而且这是可成长的，通过增加数据，扩充分类，可以更加准确，更加高效，并且可以识别更多缺陷，有助于提高打印产品质量和生产效率。

1.2.4 深度学习算法比较分析

目前各种深度学习算法各有优缺点，如

表 1-3。在 3D 打印缺陷识别中可以根据具体的场景需求和资源条件选择合适的算法。

表 1-3　3D 打印缺陷检测深度学习算法比较

算法	优点	缺点
YOLO	速度快：实时检测能力强，可以快速完成缺陷识别；精度高：能识别多种缺陷；易于部署：包含完整检测模型和实时检测功能，方便部署	数据依赖性，需大量标注数据训练；数据收集和标注繁琐；细小缺陷检测效果不佳
U-Net	分割效果好：能有效分割缺陷区域；适用于复杂场景：可处理含基板等复杂场景	模型复杂，需高计算资源和长训练时间；噪点影响分割效果
GoogLeNet	结构创新：Inception 模块提取多尺度特征 分类能力强：能准确分类不同缺陷	模型复杂，训练和推理速度慢；需高质量数据，预处理繁琐
CNN	能够进行实时检测。通过相机定期采集图像，然后采用 CNN 模型进行处理，检查打印结构的质量；历经多个轮次，可以使精度不断提高	小数据集导致模型不一致无法对产生缺陷的原因分类，即对数据要求高
强化学习	自适应调整能力强：根据反馈调整打印参数，抑制缺陷；优化全局性能：	需大量试错和迭代训练，过程复杂耗时;换环境需重新训练

上表分别对 YOLO、U-Net、GoogLeNet、CNN 还有强化学习算法进行优缺点对比分析。可以看出 U-Net 算法可以有效地对缺陷出现的区域进行分割，对复杂的检测问题处理效果好，适合处理复杂场景，但是这种模型复杂度高，这就要占用很多的计算资源和训练时间，而且噪点对这种算法的分割效果影响很大，不适合用于 FDM 缺陷检测。GoogLeNet 算法可以快速地分辨多种缺陷，却也存在模型复杂的问题，要求更多的数据资源和更长的训练时间，用于 3D 打印缺陷检测的性价比不高。CNN 算法可以应用于实时检测和多种缺陷检测，但是 CNN 算法在数据质量低时对缺陷分类能力低。YOLO（You Only Look Once）算法因为结构合理，在实时检测和复杂场景中具有独特的检测优势，具有检测速度快、效率高、易于部署等特点，十分适用于对检测速度和精度要求较高的 FDM 3D 打印缺陷检测场景，如下图 1-2 思维导图所示。多方对比，最终选用 YOLO 算法作为本设计实现缺陷实时检测目标的工具。

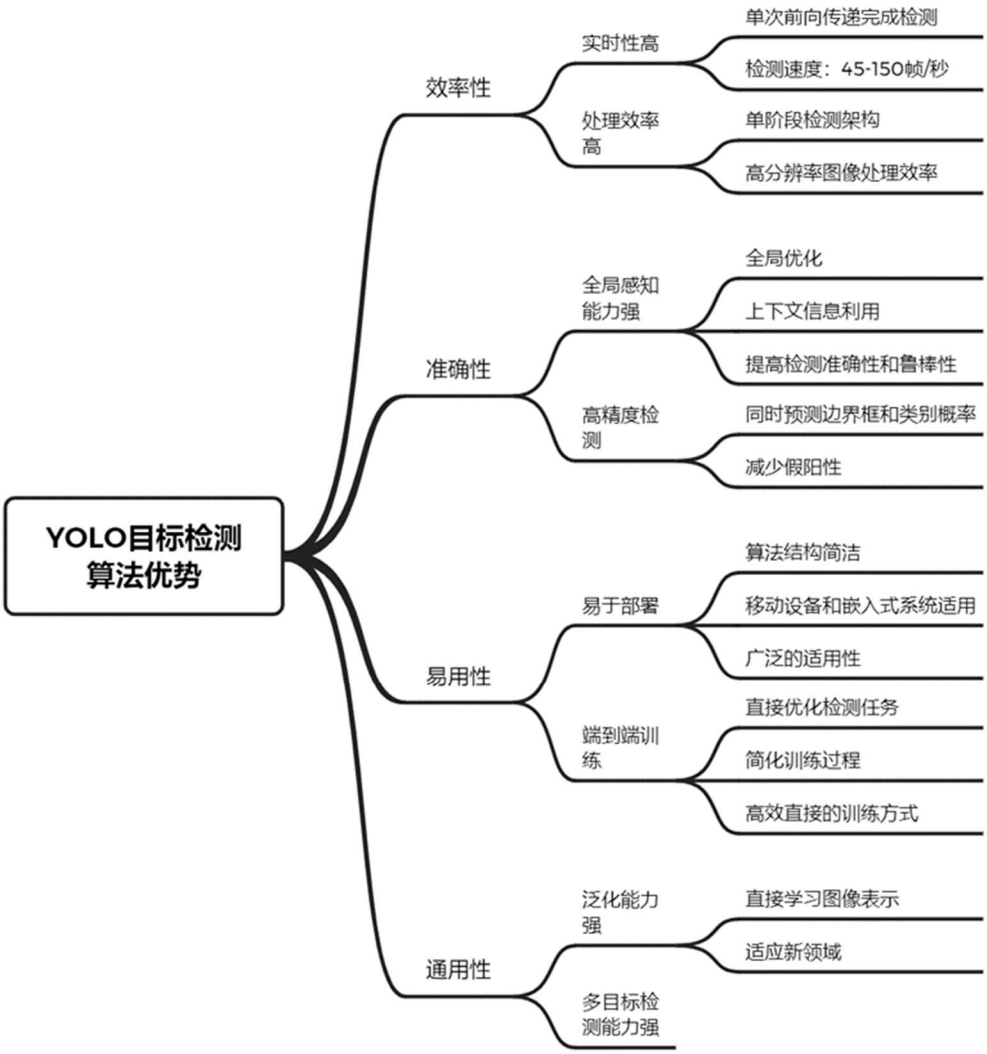


图 1-2 Yolo 目标检测算法优势思维导图

1.3 研究内容与方法

1.3.1 研究内容

根据国内外研究现状来看，研究者们正从实时监测、传感器融合、机器视觉技术、深度学习、点云数据处理以及多轴检测等多个角度对 3D 打印缺陷检测技术进行研究，以提高检测的准确性和效率。从目前的文献中可以发现，目前的 3D 打印缺陷检测方法往往存在有成本高、适用范围不广、依赖个人经验、没有统一标准等问题。对于使用者来说缺少一种能够简单快捷实时分析由打印机、丝材、参数设置引起的打印缺陷，并给出相应解决方案的工具。

本文的研究内容如下：

第一章，说明研究的背景、目的及意义。检索国内外对于 FDM 3D 打印缺陷检测的研究方法和成果，取长补短，提出适用于本课题的研究方法。

第二章，选择可实现的缺陷检测方案，并设计一种可行的图像采集方案，通过

CCD 相机输入打印实时状态可以实现在打印全过程中实时采集打印数据，利用 Python 搭建完成深度学习模型。

第三章，收集打印缺陷图像数据建立数据库，导入深度学习模型训练，根据模型的表现调整参数，不断优化模型，使之达到满意的检测效果。

第四章，基于前面形成的模型开发 GUI 窗口，方便的人机交互系统可以让使用者快速理解缺陷数据。

第五章，对本文的研究内容和成果进行总结，研究思路如图 1-3 所示。

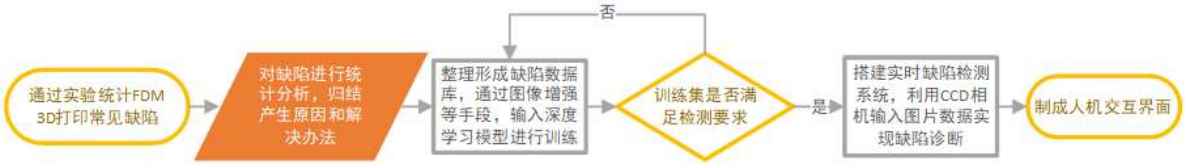


图 1-3 研究思路

1.3.2 研究方法

系统的分析 FDM 3D 打印机在打印过程中的常见缺陷，使其实现全面覆盖，结合已有的使用 FDM 3D 打印机的经验和相关文献，归纳总结 FDM 3D 打印技术在打印产品过程中出现的各类损伤缺陷。 利用在打印机上搭载 CCD 相机实现图像采集，不影响打印机正常工作且高质量采集图像，使其高效率而又不影响打印机正常工作地实现 3D 打印的全过程采集，并设计一个补光装置来完成整个采集系统的搭建。为了增强数据量，采用旋转、翻转和缩放等数据增强方式来扩充样本容量，提高算法模型的识别精度，使后续的模型训练具有足够的样本数据。基于 YOLOv10 针对几类缺陷建立深度学习算法模型，使其高精度、高效率的实现 3D 打印损伤缺陷识别，构建适合于某一类 3D 打印缺陷损伤识别的算法模型，通过训练样本数据集并不断修改训练参数，并通过验证集分析整个 3D 打印缺陷识别系统。当检测出存在打印问题时，可以给出打印问题的类型并分析产生此问题可能的原因，最后给出相应的改进意见，比如调整温度、更换材料、调整速度等等。

第2章　实时缺陷检测平台的搭建

2.1　搭建图像采集系统

想要实现实时缺陷检测，通过人力依靠肉眼观察，不仅效率低、成本高而且个人主观性强依赖于检测者的个人经验何主观判断，往往准确率无法保证。这就需要通过搭建一个图像采集系统，将图像采集后交给机器进行识别。而选择一个合适的摄像头就显得更加重要，无需接触被测物体，不用担心干扰打印过程。这个摄像头的分辨率需要满足可以分辨缺陷的基本要求，还要满足可以适配不同场景，可以安装在各种不同的打印机上进行缺陷识别，从成本上看，也不能选择价格过高的工业摄像头，增加检测成本。

摄像头的分辨率过高会导致图像数据传输慢，从而导致检测速度减慢，而过低的分辨率会使图像损失细节，导致无法正常的检出缺陷。因此，选择一款 1920*1080 200 万像素的 CCD 摄像头，可以通过 USB 连接到计算机，满足低延迟和分辨率的要求。然后通过一个支架将之安装在打印机的合适位置，使之能够在打印过程中完整拍摄到整个打印件，同时可以根据实际情况的需要，在框架上安装补光灯，以满足采集图像的亮度需求。

如下图 2-1，通过将摄像头采集到的图像数据实时经过数据传输设备发送到计算机，经过模型对比计算，从而达到识别缺陷的目的。

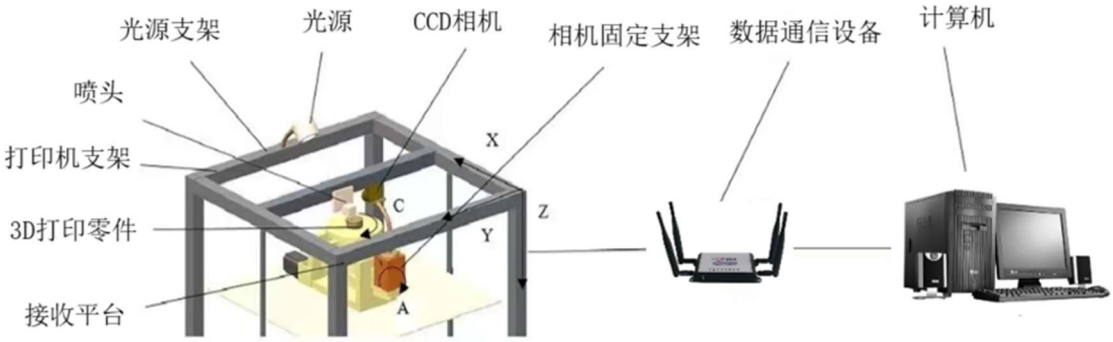


图 2-1 3d 打印平台缺陷检测系统示意图

2.2　收集整理缺陷数据

2.2.1　采集缺陷图片建立数据库

在本文 1.2.1 小节中已经对缺陷种类和成因进行了细致的分析。模型在打印过程中可能出现的缺陷多种多样，例如翘边、层间剥离、模型失附、填充不全、渗漏、表面缺陷等等。考虑到模型大小等限制，选择从众多种类缺陷中选出几种具有代表性、实际生产中易出现的几种缺陷，最终确定为意大利面式缺陷、小疙瘩、拉丝缺陷三种。

1. 意大利面式缺陷（spaghetti）

缺陷特征：打印的过程中材料形成类似意大利面条的杂乱结构。在喷嘴移动时材料没有停顿，而是持续渗出，导致打印丝拉丝或滴落，非打印区域会出现一些细丝残留，极大地影响打印件的表面质量，如图 2-2。

图 2-2 意大利面式缺陷示例图

产生原因：喷头温度设定过高而导致耗材在打印挤出后无法第一时间冷却固化，流动性过强，挤出量太多；打印平台温度设置不当，或没有粘附剂，首层没有牢牢粘在平台上，模型位移后导致后续打印也出现错位，层间粘接强度不足。

解决办法：根据材料特性调整喷头温度和打印平台温度；使用黏合剂；即时清洁喷嘴；调整熔丝挤出速度与填充速度，避免出现断丝或堆积；优化模型设计，减少薄壁和尖角设计，这些结构在打印时极易造成打印问题。

2. 小疙瘩缺陷（zits）

缺陷特征：模型表面出现不规则的凸起，分布较为零散，可能呈球状、椭球状或不规则块状，通常为两毫米左右大小，这种缺陷破坏了模型原本光滑的表面，使模型的表面不再美观，如图 2-3。

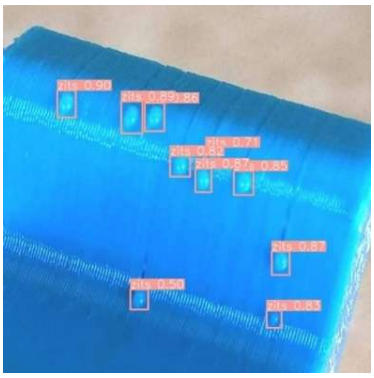


图 2-3 小疙瘩缺陷示例图

产生原因：喷头温度设置过高，在材料挤出时容易挤出过量的材料，然后在喷嘴周围堆积，随着打印头的移动，这些堆积在一起的材料就可能会黏附到模型表面，形成小疙瘩；回抽距离设置过短或回抽速度设置过慢，停止挤出材料后，还会有少量材料从喷嘴继续挤出，而在模型表面形成小疙瘩。

解决办法：调整打印温度；不使用受潮的材料，平时做好防潮处理；设置合适的回抽距离和速度。

3. 拉丝缺陷（stringing）

缺陷特征：在模型表面会出现细长的、不规则的残留耗材线，这些丝状物通常连接在模型的不同部分之间，就像蜘蛛网一样，严重影响模型的外观和表面光洁度，如图 2-4。

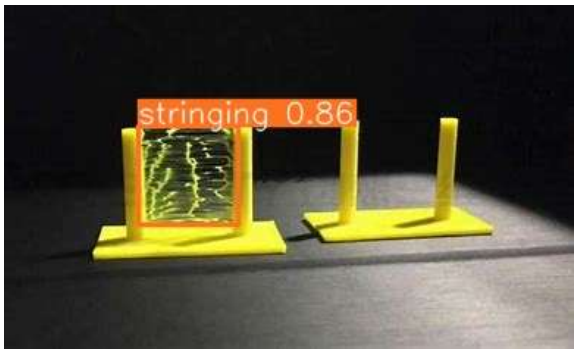


图 2-4 拉丝缺陷示例图

产生原因：在高速打印时回抽不足引发拉丝；冷却风扇转速设置太低，挤出后的材料无法及时冷却，未固化的材料极易被拉伸形成拉丝；打印速度设置过快，喷嘴无法及时地控制材料的挤出，产生拉丝；喷嘴的堵塞物在工作中被冲开，大量熔融材料会在短时间内挤出，从而产生拉丝。

解决办法：保持打印环境温度稳定，避免外界气流等因素的干扰；合理降低打印速度，调整风扇转速，控制材料冷却速度；定期清理或更换喷嘴。

2.2.2 数据预处理与增强

在数据收集阶段总共收集 5848 张缺陷图片（都属于上一节分析的三种缺陷），然后将这些图片按 8:1:1 的比例随机分配为训练集、验证集和测试集。

在数据收集和划分完成后，要对数据进行预处理，因为这么多图像数据大小规格，亮度、方向等等参数都不同，所以首先要对其进行处理以保证数据集的质量，第一步对所有的图像进行方向矫正，然后将所有图像调整为同一个分辨率，这里选择通过拉伸裁切调整为 640×640 像素，以此保证所有的图像都可以符合模型的输入要求，在保留有图像特征的同时还能保证模型的处理速。

由于只固定了一台摄像头，基本位于同一位置拍摄图像，经研究发现由于打印设置，所有拍摄的图像中，打印缺陷往往集中在同一个区域，即使这样子方便模型学习同一位置的特征缺陷，但也会导致模型对其他区域的检测效果下降甚至为零，因此就需要采取数据增强的手段扩充数据集，以达到模型泛化检测的目的。这些数据增强方法随机调色、调整亮度、随机旋转、翻转、裁剪、缩放、拼接等等，可以模拟不同方向、不同视角、不同环境下的拍摄表现，从而提升模型检测能力。

2.3 基于 YOLOv10 搭建模型

2.3.1 Yolov10 的网络架构

YOLOv10 的网络结构由“Backbone”、“Head”和“Predict Postprocess”组成，具体结构如下图 2-5。

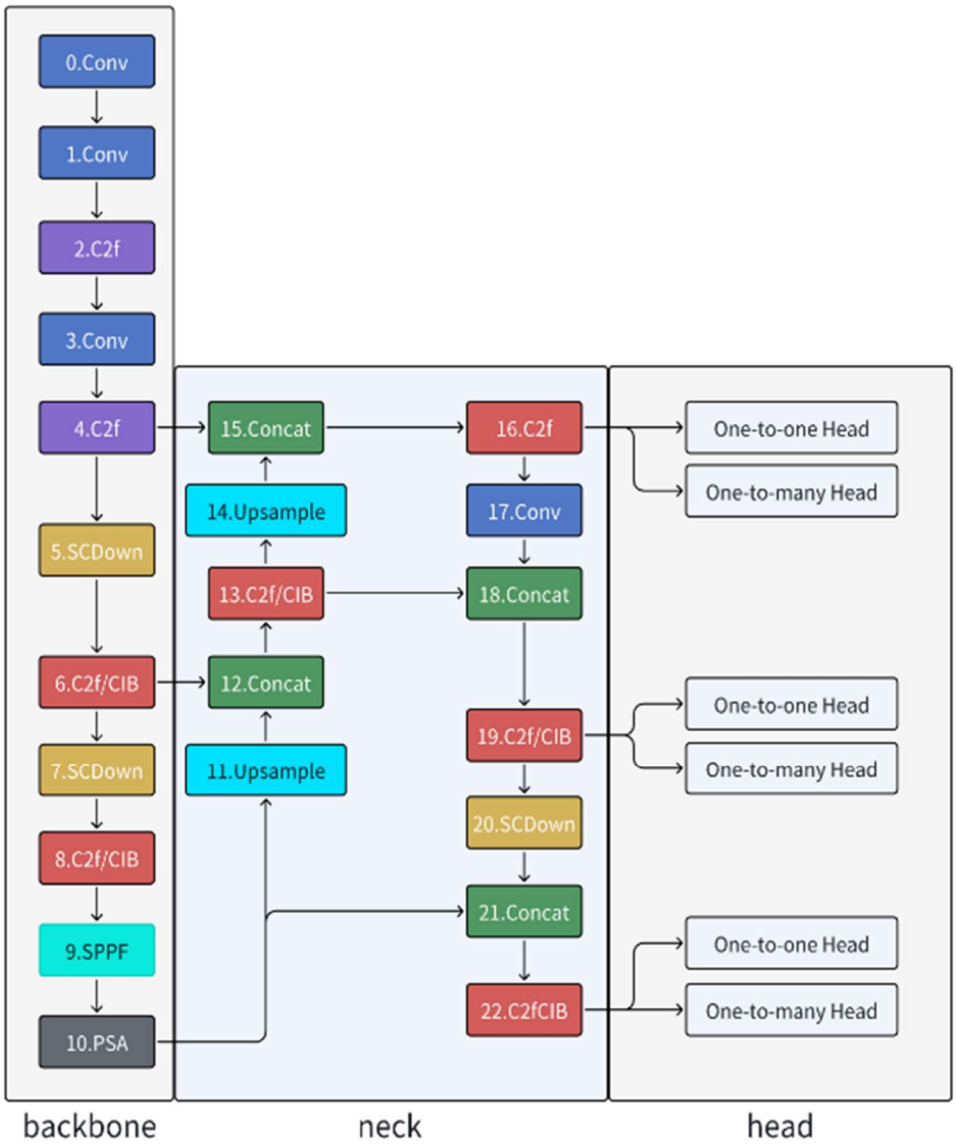


图 2-5 Yolov10 网络结构图

V10 的主干网络采用和 V8 一样的 C2f 基础结构，第七步引入的模块 SCDown 可以让特征提取的过程更加准确，这个模块甚至把卷积细化到了 1×1 的程度，大大减轻了模型的运算压力。

网络头部采用"一对多头"与"一对一头"双结构，在训练的时候，两者共同学习训练，在推理时只采用一对一头的结构，结合了两种结构的优点，模型更加高效。

2.3.2 检测原理

YOLO 是一系列的目标检测算法，其网络架构在不断发展和改进，现在已经有了 v12 的版本，由 24 个卷积层和 2 个全连接层组成。具体网络架构如图 2-6。

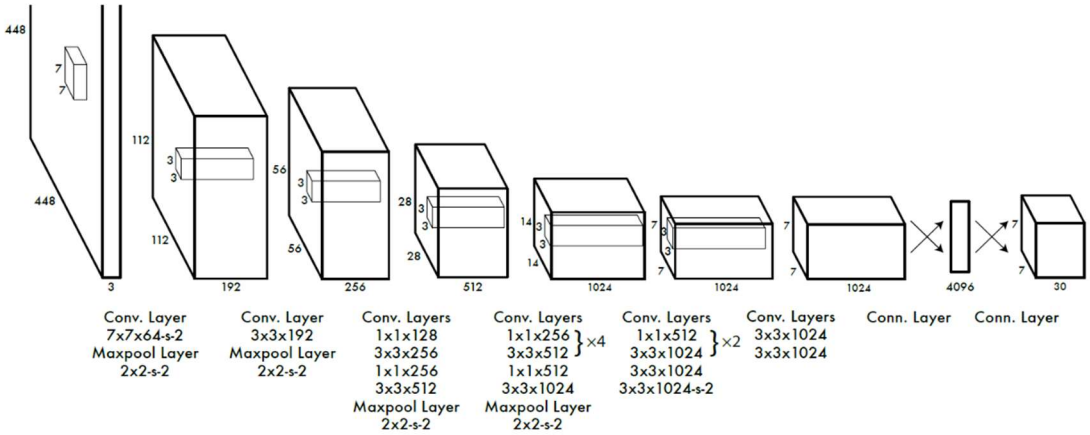


图 2-6 YOLO 网络架构

当一张图片输入进 yolo 模型后，首先会被划分为 S×S 的一个网格图形，如图 2-7 然后模型对于每一个网格都会预测 B 个边框，也就是说总共可得 S×S×B 个边界框，一个边框包含置信度和类别概率的信息。模型筛除冗余窗口后输出的张量就是 S×S×(5*B+C) ，对于一个分类问题，每个单元格预测出 C 个类别的类别概率值，其实这个概率其实就是预测目标在各个边界框置信度下的条件概率，若有 20 个类别，即当 S=7，B=2，C=20 时，可得最终的预测结果为 7×7×30 大小的一个张量，正和图 2-6 的输出一样。

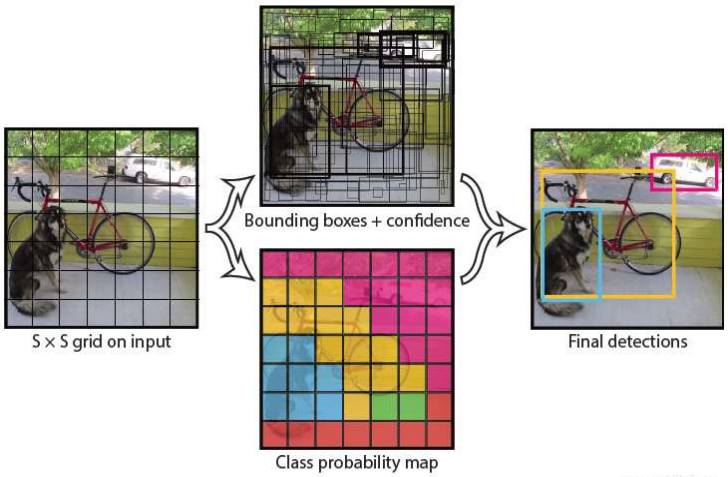


图 2-7 YOLO 检测原理

边界框的准确度则可以用预测框与实际框的 IOU（交并比）来表征，公式如下。

$$IOU = \frac{预测框与实际框的交集}{预测框与实际框的并集} \tag{2-1}$$

利用交并比可以量化判断预测框的预测效果，将上图中对狗狗、自行车、轿车的无数黑色预测框简化为一个最优的框。

第3章 检测模型训练

3.1 网络训练

3.1.1 激活函数的选择

神经网络是模拟人类神经元产生的，就好像前一个神经元连接到下一个神经元一样，激活函数就是这个传递信号的中介，经过激活函数的处理才会作用于下一层，从一个节点到另一个节点。

Yolo 使用的激活函数为 leaky rectified linear，是 ReLU（Rectified Linear Unit）激活函数的改进版本，改进主要是为了解决“神经元死亡”问题，就是说这个函数因为某些原因“死了”，不再发挥作用。

Leaky ReLU 的公式为：

$$f(x)=\begin{cases} x, & x<0 \\ \alpha x, & x\geq 0 \end{cases} \tag{3-1}$$

其中：α 是一个超参数（通常取较小的值，比如 0.01 ），这就是解决神经元死亡问题的关键。

与其他激活函数进行对比，如表 3-1：

表 3-1 激活函数对比表

激活函数	负区间处理	优点	缺点
ReLU	输出 0（梯度为 0）	计算高效稀疏激活	神经元死亡问题
Leaky ReLU	输出 αx（梯度=α）	缓解神经元死亡	需手动调参（α）
PReLU	输出 αx	自适应学习负区间斜率	可能过拟合
ELU	指数平滑负值（接近-1）	加速收敛	计算复杂度略高

3.1.2 损失函数的选择

损失函数（Loss Function）的作用在模型训练、优化、验证几个过程中都有体现，本质上是通过这个函数把预测值和真实值之间的差异通过数值量化表现出来，可以帮助模型通过调整参数使得损失函数达到最小值，此时的模型性能能够被不断优化，对于机器学习和深度学习来说是一个非常重要的函数。

对于不同的任务，往往有不同的常用损失函数，如表 3-2：

表 3-2　常用损失函数表

任务类型	常用损失函数	特点
回归	均方误差（MSE）	对异常值敏感， 梯度随误差增大而增大
二分类	二元交叉熵 （Binary Cross-Entropy）	适合概率输出， 优化类别概率分布
多分类	分类交叉熵 （Categorical Cross-Entropy）	结合 Softmax 输出， 处理多类别概率分布
目标检测	Focal Loss	缓解类别不平衡
GAN	最小二乘损（LSGAN）	稳生成更逼真的样本

YOLO 有三大损失函数，分类损失函数(cls_loss)、边界框函数(box_loss)和分布损失函数(dfl-loss)。

1.分类损失

分类损失属于上表中的多分类问题，应该使用交叉熵损失来计算，通过最小化这个函数，模型就可以对多个类别进行区分，比如本研究就需要对意大利面式缺陷、小疙瘩缺陷和拉丝缺陷进行区分。

2.边界框损失

这个损失函数包含两个部分，一个是中心坐标的损失，一个式宽高的损失，目的是使预测框的中心和宽高都要和真实框相近。可以使模型准确的检测出被测目标的位置和大小。

3.分布损失

由每种类别的样本存在的数量之间往往存在很大差别，这会导致模型对某一类的学习效果好，这个函数可以通过调整权重来平衡类别差异，可以提高整体检测性能。

3.2　调整检测模型

3.2.1　调整训练参数

一个深度学习模型可以调节优化的参数有很多，包括优化器、学习率、数据增强、模型结构等等，在模型调参时，应该遵循一个先后顺序，因为不同的参数对模型改进的效果影响大小不同，由大到小为学习率、批量大小、数据增强、损失权重，最后是其他参数，而参数调整后也要关注模型的指标变化，以便判断模型是否是正优化，为后续的优化方向提供参考。mAP@0.5、训练/验证损失曲线和梯度幅值都是很好的可供判断的指标，需要重点关注。

在模型训练中适当调整参数可以使模型更高效，不同参数对比如表 3-3。

表 3-3　YOLO 模型训练常用可调参数一览表

类别	名称	作用	调整建议
优化器	学习率（**lr0**）	控制参数更新步长，影响收敛速度与稳定性	配合余弦退火/分段衰减策略（如每 30 epoch 衰减 10%）
	权重衰减（**weight_decay**）	L2 正则化，防止过拟合	范围 **1e-4~1e-6**，与学习率平衡
数据增强	色调（**hsv_h**）	调整图像色调多样性	增强范围不超过 **0.1**，避免颜色失真
	饱和度（**hsv_s**）	调整图像饱和度	同色调调整
	亮度（**hsv_v**）	调整图像亮度	同色调调整
	旋转（**degrees**）	增加图像旋转多样性	小目标检测任务建议不超过 **30** 度
	缩放（**scale**）	模拟不同尺度目标	避免极端缩放（如 **0.1** 或 **2.0**）
训练过程	批量大小（**batch_size**）	影响梯度稳定性和显存占用	设为 GPU 显存允许的最大值（推荐 **8** 的倍数）
	训练轮数（**epochs**）	控制训练时长与过拟合风险	配合早停（**patience=5~20**）动态调整
模型结构	锚框（**anchors**）	预设检测框尺寸，影响初始化效果	通过 K-means 聚类自定义（需匹配数据集目标尺寸）
	分类损失权重（**cls**）	控制分类损失的贡献	类别不平衡时适当提高
	定位损失权重（**box**）	控制边界框回归的贡献	强调定位精度时微调至 **5.0~10.0**
硬件参数	混合精度训练（**amp**）	加速训练并节省显存	显存不足时强制开启，精度敏感任务谨慎使用
	多 GPU 训练（**device**）	加速训练进程	指定 GPU 编号（如 **0，1，2，3**），提升数据加载效率

3.2.2　性能对比

(1) Precision（精确率）

精确率就是正确识别的正类样本数量和所有被模型预测为正类的样本数之间的

比例，公式如下：

$$precision = \frac{TP}{TP + FP} \tag{3-2}$$

此处 TP（True Positive）是本身为正且模型识别为正类的样本数量，FP（False Positive）则指本身为负但被错误判别为正类的样本数。

观察图 3-1 中 YOLOv10n 和 YOLOv10m 的精确率随 Epoch（训练轮次）增加的曲线，可以看出这两个原模型的曲线非常不稳定一直在上下大幅波动，40 轮之后稳定在一个 0.2 的准确率左右，由此可见模型设置的原始学习率偏高，造成模型参数更新速度太快，样本批次一变就会让模型准确率急剧波动，于是尝试通过降低模型学习率和适当扩大 Batch Size（批次大小）来调整参数的更新速率。

调整后的模型精确率曲线如图中蓝色线条所示，随着训练轮次增加新模型的精确率呈现稳步上升趋势并渐趋平稳，没有出现上下大幅震荡，最后的表现明显优于之前的两个模型，这种曲线形态说明模型在训练中逐步学习到有效特征，即检测 FDM 3D 打印缺陷的能力大大提高。

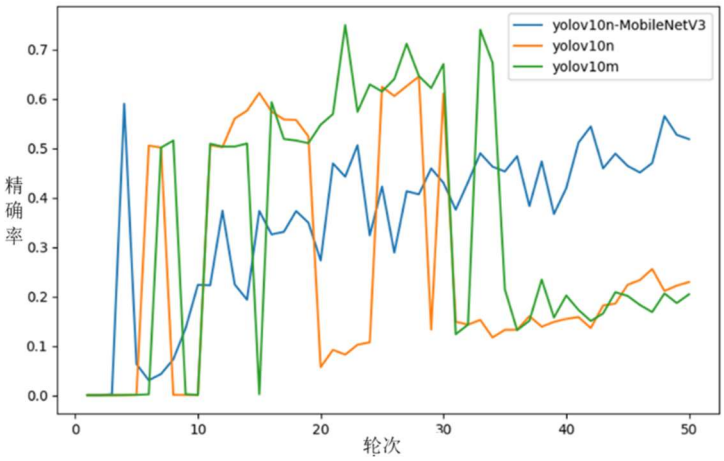


图 3-1 模型精确率增长率对比图

(2) Recall（召回率）

召回率代表真实正样本占预测出正样本的比例，公式如下：

$$recall = \frac{TP}{TP + FN} \tag{3-3}$$

FN（False Negative）表示被错误归为负类的正类样本数量。

观察图 3-2 的曲线变化趋势，三种颜色的曲线都走的是震荡上升的趋势，而蓝色线条代表的改进后模型在召回率表现上显著优于 YOLOv10n 和 YOLOv10m 模型，到后面可以稳定维持在改进前模型的两倍有余。召回率随训练轮次增加而上升说明这个模型正逐步学习到可以区分正类和负类的关键特征，模型的误判率稳步下降，这样的模型能够更可靠地识别出打印件表面的各类缺陷。

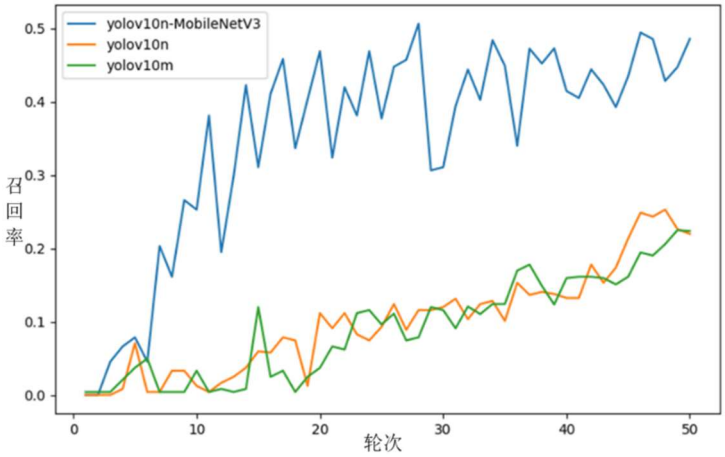


图 3-2 模型召回率对比图

(3) mAP@50（mean Average Precision at IoU=50%）

AP（Average Precision）翻译过来为平均精度，mAP（mean Average Precision）翻译为平均精确率的平均，“@50”指的是 IOU 达到 50% 时所对应的平均精确率取值，IOU 的定义和计算公式在本文的 2.3.2 节中有详细描述。

观察图 3-3 mAP@50 随训练轮次增加的变化曲线，其实和模型召回率对比图展现的效果类似，曲线震荡上行原始的两个模型的 mAP@50 值随着训练轮次增加增速，缓慢不及预期，说明原模型没有很好的学习目标位置和分类的特征。在采用强化数据增强手段对模型加以改进后，可以看到蓝色曲线代表的模型的 mAP@50 值随训练轮次提升的速率显著加快，新模型的学习性能大大提高最后可达原模型的三倍。

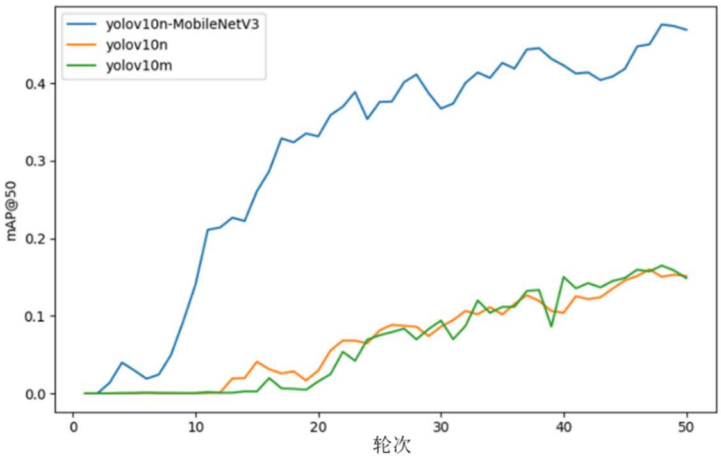


图 3-3 模型 mAP@50 值对比图

3.3 检测功能检验

(1) 收敛性分析

观察图3-4前三列所代表损失函数的变化趋势，三者的曲线都逐步下降趋于收敛，预测框损失收敛情况不错，而分布损失在 100 轮时还没收敛，需要调整数据集的数据分布，分类损失很早就下降的很低，要判断是否存在过拟合的问题。这几张图都没有出现震荡的问题，说明整体趋势向好。

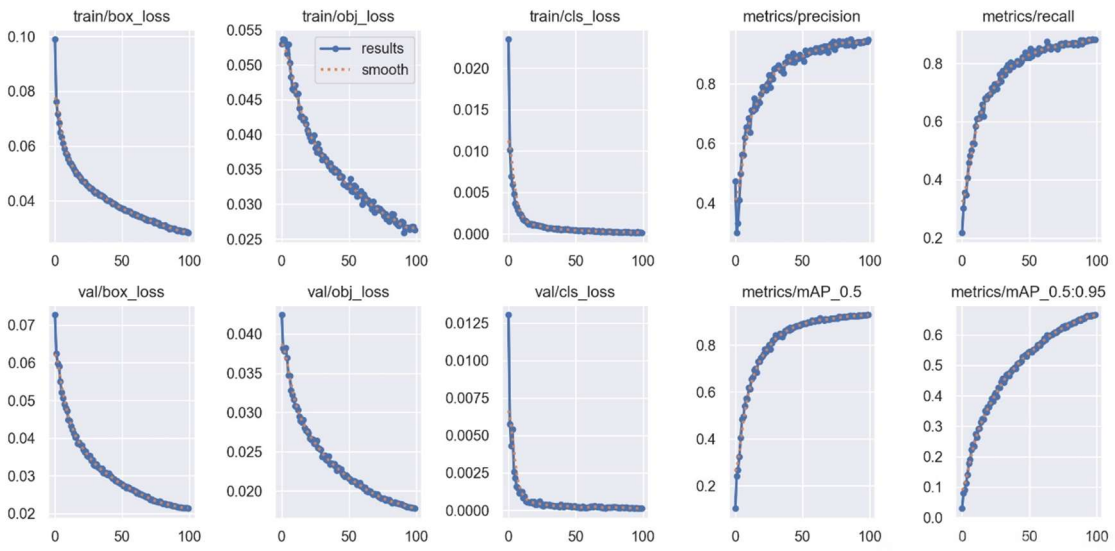


图 3-4 收敛性分析

(2) PR 曲线（Precision-Recall Curve）

PR 曲线是由精确率和召回率共同构成的一对数组（Precision，Recall）。曲线越趋于正方形表明模型在高召回率条件下仍能维持高精确率，平衡点（BEP）为曲线上 P=R 时的取值，这个值越高，说明模型性能越好，从图 3-5 中可以清晰看到三种缺陷和所有分类 mAP@50 的 BEP 分别为 0.975,0.953,0.858,0.929。

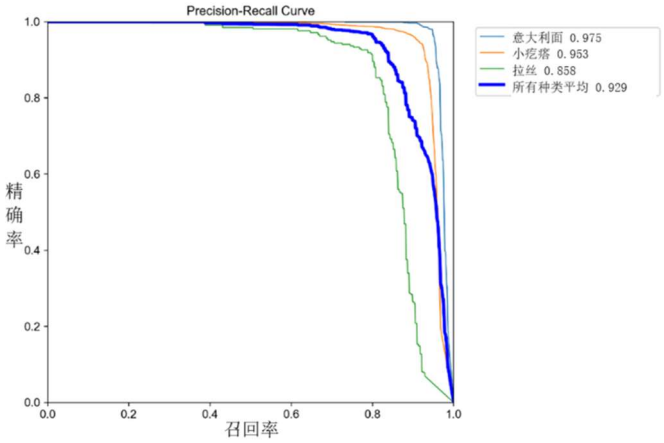


图 3-5 模型 PR 曲线图

(3) 精确率-置信度曲线（Precision-Confidence Curve）

如下图 3-6，横轴为模型的预测置信度（Confidence），即模型把样本预测为正类的概率。纵轴为精确率，将模型预测的置信度从高到低排序，并计算每个阈值下的精确率，连点成线就是上图。图中可以看到，当置信度大于 0.934 时，所有分类的精确率均可达到 1，若高置信度对应的精确率高，说明模型对“确信的预测”结果可信。若低置信度对应的精确率低，说明模型对“不确定的预测”结果不可靠。

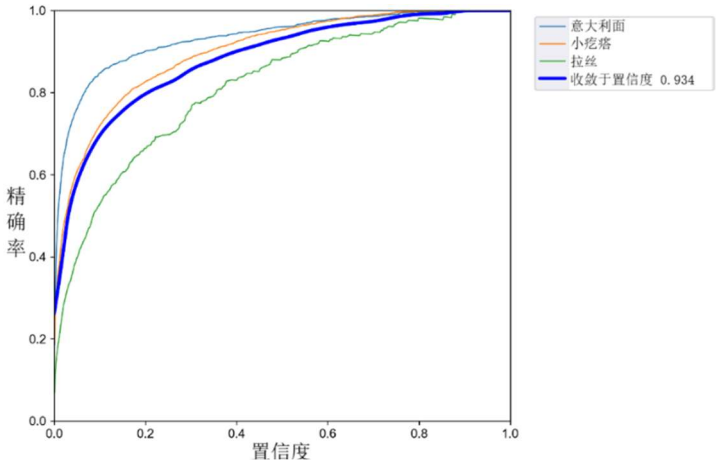


图 3-6 模型精确率-置信度曲线图

(4) 召回率-置信度曲线（Recall-Confidence Curve）

观察图 3-7 的曲线，可以看出当置信度的阈值取值为 0 时，即模型将所有样本视为正类时，召回率为 0.95。即此时的所有真实正样本中 95%能够被正确预测，从图中看模型在高置信度下召回率仍较高，说明其能比较准确地识别正样本，如图中可看出模型在置信度阈值大于 0.6 后下降趋势变大，大于 0.8 后急速下降，之前的曲线都相对比较平缓，在检测时要合理选择置信度阈值，尽量使召回率在较高的位置，减少误报处理成本。

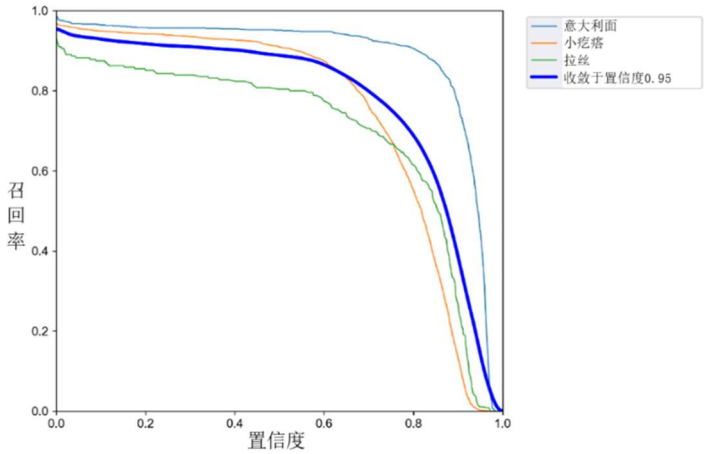


图 3-7 模型召回率-置信度曲线图

(5) F1 分数-置信度曲线（F1-Confidence Curve）

如图 3-8，横轴仍然为模型的预测置信度，纵轴为 F1 分数，是综合反映精确率和召回率的指标，计算公式为：

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{3-4}$$

从公式上不难看出 F1 分数在精确率和召回率之间寻求平衡，避免单一指标的片面性。整个曲线类似一个凸台，存在最大 F1 分数，且在合理置信度范围内保持较高值。观察上图，在置信度 0.2~0.8 的范围内，F1 分数都维持在一个较高值，并且可以从右侧的极大值得出所有分类的曲线峰值 0.91 对应的阈值置信度为 0.561，即峰值

F1=0.91 对应阈值=0.561。

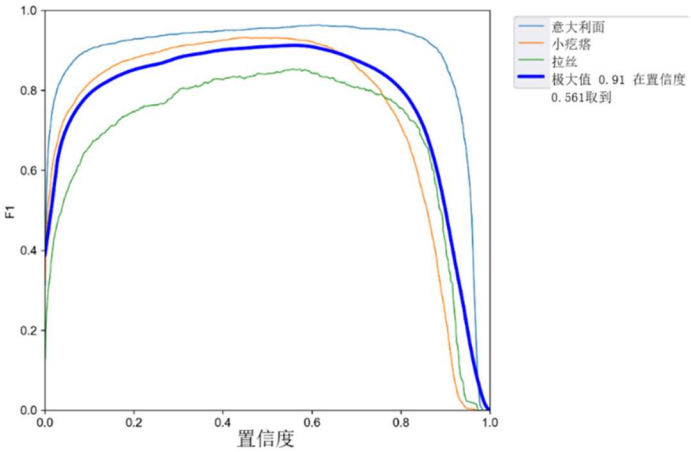


图 3-8 模型 F1 分数-置信度曲线图

置信度从 0 到 0.2 时，在召回率缓慢下降和精确率迅速上升的情况下反映到 F1 分数上就表现为其取值快速上升，然后在置信度 0.2 至 0.8 之间曲线比较平缓，置信度高于 0.8 后由于精确率缓慢上升和召回率迅速下降 F1 分数同样表新出快速下降趋势。图中标出使精确率和召回率达到极大值的最佳阈值是 0.561，具体使用模型时可以作为参考，根据想要达到的检测效果适当上下调整置信度取值。

第4章　设计人机交互界面

4.1 界面设计

为了方便缺陷检测系统与操作人员之间的信息交互开发了图形用户界面（Graphical User Interface，GUI）用以展示缺陷识别结果，GUI 比普通的命令行交互更加直观简便，下图 4-1 展示了界面制作成果。

图 4-1 人机交互界面图

1.将窗口命名为“ YOLOv10 3D 打印缺陷检测系统”，并且在窗口顶部中央位置设置了相同的标题名称，同时为了使用者能在第一时间清楚了解到本系统的功能特意调整了标题的字号和字体使标题更加醒目。

2. 将整个界面划分为文件导入、检测结果、位置信息、图像显示、操作，五个板块，不同板块实现不同功能

3. 在“文件导入”框中创建标签和输入框，用于导入检测图像。从上至下分为图片、视频和摄像头，点击对应的输入框可以选择相应的文件导入，点击摄像头会自动识别计算机连接的摄像头，调用其图像进行识别。

4. 在“检测结果”框中，加入检测所用时间、在一张图片中检测到该缺陷的数目、并提供选择检测缺陷的目标。完成选择后，显示缺陷检测的类型、以及置信度，还有该缺陷在图片上的位置。

5. 在“图像显示”框中显示正在被检测的图像，并用不同颜色方框直观地标出缺陷的位置，在方框上注明缺陷的种类。

6. 在“位置信息”框中，显示每一个被检测到缺陷的类别、置信度、坐标位置、

所属图片的文件路径，并按先后顺序将检测到的每个缺陷标号，方便查找。

7. 在“操作”框中，提供“保存”和“退出”按钮，点击保存按钮会将处理后的检测图片保存到指定的文件夹中，点击退出按钮将会退出程序。

8. 在导入文件时，设置提示信息，当没有导入可用于检测的图像时，会在对应的框中以灰色文字显示“请选择图片文件”和“请选择视频文件”，告知使用者没有图像或视频用于检测，当检测不到摄像头时，会以灰色文字提示“摄像头未开启”，在导入文件之后，对应的框中显示文件路径，点击摄像头图案会打开摄像头，并在旁边以黑色字体显示“摄像头开启”，如图 4-2。

图 4-2 导入文件框

4.2 界面功能检验

在上一节检测人机交互界面完成后，需要对缺陷检测的功能进行检验，确定该缺陷检测的可靠性，以及人机交互界面的易用性，确保设计能够在实际应用中稳定运行。另外，通过检验，能够发现界面设计可能存在的一些漏洞及问题，这样能够方便对系统进行错误总结，进而进行修复和改进，提高 FDM 3D 打印缺陷检测系统的性能和精度。检验方法如下：

首先选择一张可以反映检测功能的图片，如下图 4-3，经过人工判断可以得出，左边打印的两根立柱之间存在拉丝缺陷，而右边两根立柱没有肉眼能直观看出的表面缺陷，如果缺陷检测系统功能正常，应该只检测出一出拉丝缺陷，即左侧两根立柱中间的方框区域存在缺陷。

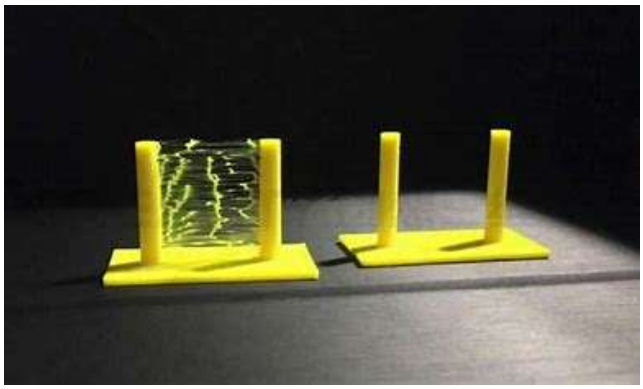


图 4-3 导入的图片

将该图片保存在文件夹中，并将文件夹命名为“ceshi”，然后在界面上选择图像导入的按钮，选择“ceshi”文件夹所在的位置，将之导入到检测系统中，如下图 4-4。

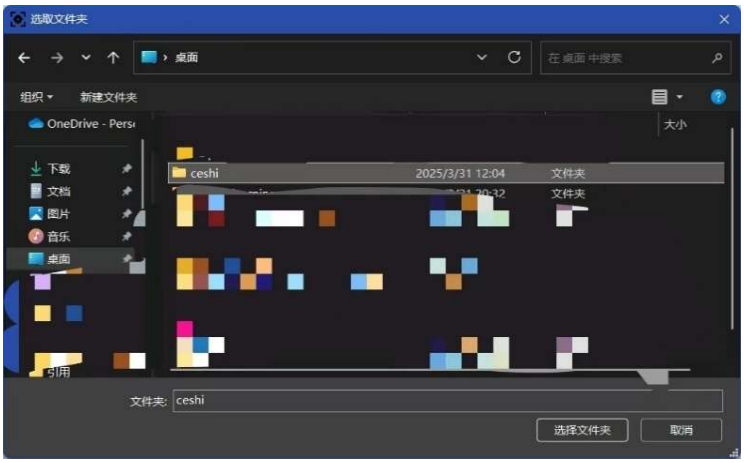


图 4-4 导入图片界面

成功导入后的界面如下图 4-5 所示，导入后的图片会直接进行检测，并显示检测用时、数目、缺陷类型、位置等等一系列缺陷信息，可以看到，检测的准确性还是不错的，只检测了一处左边的拉丝缺陷，并没有出现误检测、漏检测的现象。

图 4-5 导入图片后界面

最后点击右下角“操作”框中的保存按钮，就会将检测完的图片保存到对应文件夹中，打开文件夹找到刚检测完的图片，如下图 4-6。说明图像保存功能正常，可以正常保存缺陷图像。

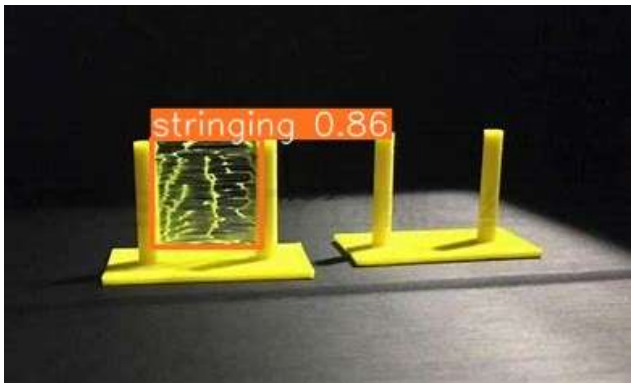


图 4-6 保存的图片

第5章　结论与展望

本文针对随着高速打印技术发展和 3D 打印工厂产能的扩大，FDM 3D 打印产品缺陷增加的问题，提出了一种基于机器视觉的 FDM 3D 打印缺陷检测从而提高打印效率的方案，通过缺陷种类分析、图像采集、图像预处理、基于 yolov10 的改进模型、设计人机交互界面等一系列方案，可以实现对 FDM 3D 打印产品进行实时的缺陷检测。

(1) 本研究创新性地提出了一种基于机器视觉技术的实时缺陷检测系统。运用 OpenCV 视觉处理引擎，基于 YOLOv10 算法构建了可以实现实时缺陷检测的框架，完成对意大利面式缺陷、小疙瘩缺陷、拉丝缺陷的毫秒级实时响应。经严格测试，模型能够在线识别小至 0.1mm 的缺陷，准确率可达 92%。

(2) 采集者需要拥有 3D 打印专业知识与和丰富的缺陷处理经验才能满足收集缺陷图像的要求，另一方面，建立一个数据集还要求收集大量数据。为克服这些难题，开展了大量的打印实验，调整切片参数以人为介入的方式打印了很多具有缺陷特征的模型全面采集多类型缺陷图像数据，建立数据集后还需要用工具标注出缺陷框和类型，这都要耗费大量的时间成本。

(3) 在最后阶段设计的人机交互界面可以实时展示缺陷检测结果和数据，让 FDM 3D 打印的缺陷检测过程更加直观便捷，辅助操作者迅速识别缺陷并进行缺陷判断与结论推理，通过降低缺陷识别的时间和人力成本从而提升 FDM 3D 打印的效率与准确性。

本研究未来还有可以改进之处，比如可以安装多个相机同时采集，同时处理，不仅可以收集更多图像数据，同时多角度的图片用于训练也能提高模型精度。可以设计一套反馈系统，使模型可以从后续的打印中自我学习，提高缺陷检测能力，可以检测更精确，更多缺陷。

综上所述，FDM 3D 打印的缺陷识别是一个复杂的过程。不仅需要了解打印过程中出现的不同类型的缺陷，还需要基于此构建模型，然后不断优化模型，最后的人机交互界面集以上为一体，将检测功能集于一身。虽然由于本文的实验数据搜集不够大量，可以检测的缺陷案例较少，但只要将其投入运行，不断搜集更多的缺陷图像，完善缺陷数据库就能使得模型不断学习进化，最终实现所有缺陷的高效率检测。

参考文献

[1] 明宪良，汤洁，党耀石，等.航天领域金属 3D 打印技术发展方向与产业化建议[J/OL].金属加工(热加工)，1-10[2025-03-20].http://kns.cnki.net/kcms/detail/11.5627.TH.20250306.1307.004.html.

[2] 田彩兰，南阳瑞.3D 打印技术发展趋势及其在商业航天上的应用[J].航天制造技术，2024，(01):64-70.

[3] 刘程琳，王沛.3D 打印技术在创新创业实践中的应用探究[J].中国现代教育装备，2025，(03):130-132.DOI:10.13492/j.cnki.cmee.2025.03.019.

[4] 邢洪亚，陈继飞.基于 FDM 的多自由度 3D 打印技术研究[J].南方农机，2025，56(01):138-141.

[5] 中国矿业大学金属材料与加工系，王延庆，张存生，等.3D 打印实用技术[M].化学工业出版社:2023，05.284.

[6] Saxon T J E . “Additive Manufacturing Technologies: 3D Printing,Rapid Prototyping and Direct Digital Manufacturing”, 2nd Edition[J]. Johnson Matthey Technology Review, 2015, 59(3):193-198.

[7] Mohamed A O ,Masood H S,Bhowmik L J. Optimization of fused deposition modeling process parameters for dimensional accuracy using I-optimality criterion[J]. Measurement, 2016, 81174-196.

[8] 张海光，宗在亮，姚远，等.基于机器视觉的多轴 3D 打印缺陷检测技术研究[J].计量与测试技术，2021，48(11):45-47+52.DOI:10.15988/j.cnki.1004-6941.2021.11.014.

[9] 张月桐，陈文亮，王向军，等.基于红外成像的 FDM 打印轨迹测量及质量评价方法[J].应用光学，2024，45(05):1008-1018.

[10] 李飞.基于点云数据和深度学习的 3D 打印表面缺陷检测研究[D].华中科技大学，2021.DOI:10.27157/d.cnki.ghzku.2021.000770.

[11] 陈旭，李筠.基于三维激光成像技术的 3D 打印表面成型缺陷自动检测[J].激光杂志，2024，45(11):214-219.DOI:10.14016/j.cnki.jgzz.2024.11.214.

[12] 王红梅.基于 AI 算法的 3D 打印缺陷检测方法研究[J].西安文理学院学报(自然科学版)，2023，26(01):25-29.

[13] Zackary Snow,Luke Scime,Amirkoushyar Ziabari, et al. Scalable in situ non-destructive evaluation of additively manufactured components using process monitoring, sensor fusion, and machine learning[J/OL]. Ad ditive Manufacturing, 2023, 78: 103817. DOI:10.1016/j.addma.2023.103817.

[14] Xiaobin W,Shuang G,Jianlan G,et al.Deep Learning-Based Integrated Circuit Surface Defect Detection: Addressing Information Density Imbalance for Industrial Application[J]. International Journal of Computational Intelligence Systems, 2024, 17(1):253.

[15] 孙伟俊.基于机器视觉的 3D 打印表面缺陷多角度在线检测技术研究[D].浙江大学，2019.

[16] 霍振浩，陈劲松，闫琦昊，等.基于机器学习的 3D 打印技术发展现状及趋势[J].造纸装备及材料，2024，53(07):77-79.

致　谢

直到写毕业论文时才发现我的大学四年生活，已经走入了尾声，回望大学生活，从刚走入校门的迷茫，到现在熟悉校园的一草一木，一次次早八走过的自强桥现在即将拆除，这四年有太多的幸运也有太多的遗憾，遇到了很多人，也遇到了很多事，他们都伴我一天天成长。

大一刚来的时候，我被分在了其他班的寝室，和隔壁班的同学混的倒挺熟，军训还找不着自己专业的队伍，跑到航工的训练方阵里训了两天，直到和一起站军姿的兄弟谈起才发现不是自己班的队伍，赶紧和教官打招呼找回自己方阵。我把军训当成大学的第一门课，努力训练，也尝试参加国护队，奈何实在无法满足训练要求，没能去成。最终成为学院的护旗手之一，扛旗走过最后一天的验收，算是一个比较好的开始。

大二的时候，有门《机械制图》的课，我觉得十分有趣，兴趣使然，塑造一个 3d 物体确实让我着迷，同时我也十分焦虑，这时候都开始卷绩点，拼竞赛，而我还不知道怎样去参加一个大创，偶然的机会，通过好友介绍了解到周蕊老师的 3d 打印团队，参观一番被一排排的打印机深深吸引，当即决定“我一定要加入这个团队”。刚进入的时候，周老师就给我了个大活——给打印机“小黑”建模，对软件建模一窍不通的我，就一边建模一边学习，建完也算是比较熟练掌握了 sw。她是我大学阶段对我最重要的人，周老师对待科研非常严谨，学术上又有丰富的学识，总是在我和其他同学迷茫没有方向的时候，为我们的困惑指点迷津，是她带我一步步学习研究。

同时，我也感谢我的师兄师姐们，他们对整个课题组的大家总是充满热情，组里充满活力。特别感谢卢帅员师兄、吴浩师兄，他们直接的指导我实验方案，教我实验的操作和流程，还有吕山子、盛旭东、裴天瑞、邱灵杰、白书博等其他同学在缺陷图像获取、日常生活等方方面面的帮助。

我还得感谢我的大学同学，我的室友们，不管是换寝室前还是换寝室之后的室友，他们是我本科四年期间陪伴我最久的人，感谢他们的陪伴，在无聊孤独时，总有同伴在身边。

虽然和高中同学各奔东西，但他们依然是我最最亲密无间的战友，哪怕常常互损，但是在我最需要帮助的时候，他们总能给出真挚的建议，帮助我闯过一次又一次的难关，希望他们都能在自己的人生轨道上越走越远。

最后，我还想感谢一下我的家人，和一直以来支持我的长辈们。他们是我求学路上最坚强的后盾，从小时候教我学习方法、为人处世、种种知识，等长大后学习的内容超出他们的能力范围以后，他们也在背后支持着我，爷爷奶奶总是偷偷告诉我，没钱了就给他打电话，也许我学到的知识对他们来说太晦涩难懂，但是他们希望我天天能吃好、睡好，过得开心。

拥有以上的种种幸运的同时，这四年也留下了种种遗憾，这四年里，没有好好的出去走走，天津也只玩了小小一部分。大一总说课多，等课少了就可以到处逛逛，然而真到了大三大四，我更希望待在学校，或是去健身房锻炼一会，当然锻炼出更明显的肌肉线条也让人挺有成就感的。总是有各种理由让自己处于一个忙碌的状态，缺少了一些放松的活力与冲动。

总之，虽有遗憾，虽能做到更好，但过去的已经过去，不必纠结，来日方长！

附录 A：程序清单

```
import time
fromPyQt5.QtWidgetsimportQApplication ,QMainWindow,QFileDialog,QMessageBox,QWidget,QHeaderView,QTableWidgetItem, QAbstractItemViewimport sysimport os
from PIL import ImageFont
from ultralytics import YOLOv10sys.path.append('UIProgram')
from UIProgram.UiMain import Ui_MainWindowimport sys
from PyQt5.QtCore import QTimer, Qt, QThread, pyqtSignal,QCoreApplicationimport detect_tools as toolsimport cv2import Config
from UIProgram.QssLoader import QSSLoader
from UIProgram.precess_bar import ProgressBar
import numpy as np
# import torch
class MainWindow(QMainWindow):
    def __init__(self, parent=None):
        super(QMainWindow, self).__init__(parent)
        self.ui = Ui_MainWindow()
        self.ui.setupUi(self)
        self.initMain()
        self.signalconnect()
        # 加载 css 渲染效果
        style_file = 'UIProgram/style.css'
        qssStyleSheet = QSSLoader.read_qss_file(style_file)
        self.setStyleSheet(qssStyleSheet)
    def signalconnect(self):
        self.ui.PicBtn.clicked.connect(self.open_img)
        self.ui.comboBox.activated.connect(self.combox_change)
        self.ui.VideoBtn.clicked.connect(self.vedio_show)
        self.ui.CapBtn.clicked.connect(self.camera_show)
        self.ui.SaveBtn.clicked.connect(self.save_detect_video)
        self.ui.ExitBtn.clicked.connect(QCoreApplication.quit)
        self.ui.FilesBtn.clicked.connect(self.detact_batch_imgs)
```

```
def initMain(self):
    self.show_width = 700
    self.show_height = 500
    self.org_path = None
    self.is_camera_open = False
    self.cap = None
    # self.device = 0 if torch.cuda.is_available() else 'cpu'
    # 加载检测模型
    self.model = YOLOv10('runs/detect/exp/weights/best.pt', task='detect')
    self.model(np.zeros((48, 48, 3)))   #预先加载推理模型
    self.fontC = ImageFont.truetype("Font/platech.ttf", 25, 0)
    self.colors = tools.Colors()
    self.timer_camera = QTimer()
    # 更新检测信息表格
    # self.timer_info = QTimer()
    # 保存视频
    self.timer_save_video = QTimer()
    # 表格
    self.ui.tableWidget.verticalHeader().setSectionResizeMode(QHeaderView.Fixed)
    self.ui.tableWidget.verticalHeader().setDefaultSectionSize(40)
    self.ui.tableWidget.setColumnWidth(0, 80)   # 设置列宽
    self.ui.tableWidget.setColumnWidth(1, 200)
    self.ui.tableWidget.setColumnWidth(2, 150)
    self.ui.tableWidget.setColumnWidth(3, 90)
    self.ui.tableWidget.setColumnWidth(4, 230)
    self.ui.tableWidget.setSelectionBehavior(QAbstractItemView.SelectRows)      #
设置表格整行选中
    self.ui.tableWidget.verticalHeader().setVisible(False)   # 隐藏列标题
    self.ui.tableWidget.setAlternatingRowColors(True)   # 表格背景交替
def open_img(self):
    if self.cap:
        # 打开图片前关闭摄像头
        self.video_stop()
        self.is_camera_open = False
        self.ui.CaplineEdit.setText('摄像头未开启')
```

```
            self.cap = None
        file_path, _ = QFileDialog.getOpenFileName(None, '打开视频', './', "Image files
(*.avi *.mp4 *.jepg *.png)")
        if not file_path:
            return None
        self.org_path = file_path
        self.ui.VideolineEdit.setText(file_path)
        return file_path
    def video_start(self):
        # 删除表格所有行
        self.ui.tableWidget.setRowCount(0)
        self.ui.tableWidget.clearContents()
        # 清空下拉框
        self.ui.comboBox.clear()
        # 定时器开启,每隔一段时间,读取一帧
        self.timer_camera.start(1)
        self.timer_camera.timeout.connect(self.open_frame)
    def tabel_info_show(self, locations, clses, confs, path=None):
        path = path
        for location, cls, conf in zip(locations, clses, confs):
            row_count = self.ui.tableWidget.rowCount()   # 返回当前行数(尾部)
            self.ui.tableWidget.insertRow(row_count)   # 尾部插入一行
            item_id = QTableWidgetItem(str(row_count+1))   # 序号
            item_id.setTextAlignment(Qt.AlignHCenter | Qt.AlignVCenter)
            item_path = QTableWidgetItem(str(path))   # 路径
            # item_path.setTextAlignment(Qt.AlignHCenter | Qt.AlignVCenter)
            item_cls = QTableWidgetItem(str(Config.CH_names[cls]))
            item_location = QTableWidgetItem(str(location)) # 目标框位置
            # item_location.setTextAlignment(Qt.AlignHCenter | Qt.AlignVCenter)
            self.ui.tableWidget.setItem(row_count, 0, item_id)
            self.ui.tableWidget.setItem(row_count, 1, item_path)
            self.ui.tableWidget.setItem(row_count, 2, item_cls)
            self.ui.tableWidget.setItem(row_count, 3, item_conf)
            self.ui.tableWidget.setItem(row_count, 4, item_location)
        self.ui.tableWidget.scrollToBottom()
```

```
def video_stop(self):
    self.cap.release()
    self.timer_camera.stop()
    # self.timer_info.stop()
def open_frame(self):
    ret, now_img = self.cap.read()
    if ret:
        # 目标检测
        t1 = time.time()
        results = self.model(now_img)[0]
        t2 = time.time()
        take_time_str = '{:.3f} s'.format(t2 - t1)
        self.ui.time_lb.setText(take_time_str)
        location_list = results.boxes.xyxy.tolist()
        self.location_list = [list(map(int, e)) for e in location_list]
        cls_list = results.boxes.cls.tolist()
        self.cls_list = [int(i) for i in cls_list]
        self.conf_list = results.boxes.conf.tolist()
        self.conf_list = ['%.2f %%' % (each * 100) for each in self.conf_list]
        total_nums = len(location_list)
        cls_percents = []
        for i in range(3):
            if total_nums!= 0 :
                res = self.cls_list.count(i) / total_nums
            else :
                res=0
            cls_percents.append(res)
        self.set_percent(cls_percents)
        now_img = results.plot()
        # 获取缩放后的图片尺寸
        self.img_width, self.img_height = self.get_resize_size(now_img)
        resize_cvimg = cv2.resize(now_img, (self.img_width, self.img_height))
        pix_img = tools.cvimg_to_qpiximg(resize_cvimg)
        self.ui.label_show.setPixmap(pix_img)
        self.ui.label_show.setAlignment(Qt.AlignCenter)
```

```
            # 目标数目
            target_nums = len(self.cls_list)
            self.ui.label_nums.setText(str(target_nums))
            # 设置目标选择下拉框
            choose_list = ['全部']
            target_names = [Config.names[id] + '_' + str(index) for index, id in enumerate(self.cls_list)]
            choose_list = choose_list + target_names
            self.ui.comboBox.clear()
            self.ui.comboBox.addItems(choose_list)
            if target_nums >= 1:
                self.ui.type_lb.setText(Config.CH_names[self.cls_list[0]])
                self.ui.label_conf.setText(str(self.conf_list[0]))
                self.ui.label_xmin.setText(str(self.location_list[0][0]))
                self.ui.label_ymin.setText(str(self.location_list[0][1]))
                self.ui.label_xmax.setText(str(self.location_list[0][2]))
                self.ui.label_ymax.setText(str(self.location_list[0][3]))
            else:
                self.ui.type_lb.setText('')
                self.ui.label_conf.setText('')
                self.ui.label_xmin.setText('')
                self.ui.label_ymin.setText('')
                self.ui.label_xmax.setText('')
                self.ui.label_ymax.setText('')
        self.tabel_info_show(self.location_list,self.cls_list,self.conf_list,path=self.org_path)
        else:
            self.cap.release()
            self.timer_camera.stop()
    def vedio_show(self):
        if self.is_camera_open:
            self.is_camera_open = False
            self.ui.CaplineEdit.setText('摄像头未开启')
        video_path = self.get_video_path()
        if not video_path:
            return None
```

```
        self.cap = cv2.VideoCapture(video_path)
        self.video_start()
        self.ui.comboBox.setDisabled(True)
    def camera_show(self):
        self.is_camera_open = not self.is_camera_open
        if self.is_camera_open:
            self.ui.CaplineEdit.setText('摄像头开启')
            self.cap = cv2.VideoCapture(1)
            self.video_start()
            self.ui.comboBox.setDisabled(True)
        else:
            self.ui.CaplineEdit.setText('摄像头未开启')
            self.ui.label_show.setText('')
            if self.cap:
                self.cap.release()
                cv2.destroyAllWindows()
            self.ui.label_show.clear()
    def get_resize_size(self, img):
        _img = img.copy()
        img_height, img_width , depth= _img.shape
        ratio = img_width / img_height
        if ratio >= self.show_width / self.show_height:
            self.img_width = self.show_width
            self.img_height = int(self.img_width / ratio)
        else:
            self.img_height = self.show_height
            self.img_width = int(self.img_height * ratio)
        return self.img_width, self.img_height
    def save_detect_video(self):
        if self.cap is None and not self.org_path:
            QMessageBox.about(self, '提示', '当前没有可保存信息,请先打开图片或
视频！')
            return
        if self.is_camera_open:
            QMessageBox.about(self, '提示', '摄像头视频无法保存!')
```

```
            return
        if self.cap:
            res = QMessageBox.information(self, '提示', '保存视频检测结果可能需要较长时间,请确认是否继续保存？',QMessageBox.Yes | QMessageBox.No , QMessageBox.Yes)
            if res == QMessageBox.Yes:
                self.video_stop()
                com_text = self.ui.comboBox.currentText()
                self.btn2Thread_object=btn2Thread(self.org_path,self.model, com_text)
                self.btn2Thread_object.start()
            self.btn2Thread_object.update_ui_signal.connect(self.update_process_bar)
            else:
                return
        else:
            if os.path.isfile(self.org_path):
                fileName = os.path.basename(self.org_path)
                name , end_name= fileName.rsplit(".",1)
                save_name = name + '_detect_result.' + end_name
                save_img_path = os.path.join(Config.save_path, save_name)
                # 保存图片
                cv2.imwrite(save_img_path, self.draw_img)
QMessageBox.about(self, '提示', '图片保存成功!\n 文件路径:{}'.format(save_img_path))
                name, end_name = file_name.rsplit(".",1)
                save_name = name + '_detect_result.' + end_name
                save_img_path = os.path.join(Config.save_path, save_name)
                results = self.model(full_path)[0]
                now_img = results.plot()
                # 保存图片
                cv2.imwrite(save_img_path, now_img)
QMessageBox.about(self,'提示','图片保存成功!\n 文件路径:{}'.format(Config.save_path))
    def update_process_bar(self,cur_num, total):
        if cur_num == 1:
            self.progress_bar = ProgressBar(self)
            self.progress_bar.show()
        if cur_num >= total:
```

```
                self.progress_bar.close()
                QMessageBox.about(self, '提示', '视频保存成功!\n 文件在{}目录下
'.format(Config.save_path))
                return
            if self.progress_bar.isVisible() is False:
                self.btn2Thread_object.stop()
                return
            value = int(cur_num / total *100)
            self.progress_bar.setValue(cur_num, total, value)
            QApplication.processEvents()
        def set_percent(self, probs):
            items = [self.ui.progressBar, self.ui.progressBar_2, self.ui.progressBar_3]
            labels = [self.ui.label_20, self.ui.label_21, self.ui.label_22]
            prob_values = [round(each * 100) for each in probs]
            label_values = ['{:.1f}%'.format(each * 100) for each in probs]
            for i in range(len(probs)):
                items[i].setValue(prob_values[i])
                labels[i].setText(label_values[i])
class btn2Thread(QThread):
    update_ui_signal = pyqtSignal(int,int)
    def __init__(self, path, model, com_text):
        super(btn2Thread, self).__init__()
        self.org_path = path
        self.model = model
        self.com_text = com_text
        # 用于绘制不同颜色矩形框
        self.colors = tools.Colors()
        self.is_running = True    # 标志位,表示线程是否正在运行
    def run(self):
                cap = cv2.VideoCapture(self.org_path)
        # 设置需要保存视频的格式“xvid”
        # 该参数是 MPEG-4 编码类型,文件名后缀为.avi
        fourcc = cv2.VideoWriter_fourcc(*'XVID')
        # 设置视频帧频
        fps = cap.get(cv2.CAP_PROP_FPS)
```

```
        # 设置视频大小
        size=(int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)),
int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT)))
        fileName = os.path.basename(self.org_path)
        name, end_name = fileName.split('.')
        save_name = name + '_detect_result.avi'
        save_video_path = os.path.join(Config.save_path, save_name)
        out = cv2.VideoWriter(save_video_path, fourcc, fps, size)
        prop = cv2.CAP_PROP_FRAME_COUNT
        total = int(cap.get(prop))
        print("[INFO] 视频总帧数：{}".format(total))
        cur_num = 0
        # 确定视频打开并循环读取
        while (cap.isOpened() and self.is_running):
            cur_num += 1
            print('当前第{}帧,总帧数{}'.format(cur_num, total))
            ret, frame = cap.read()
            if ret == True:
                # 检测
                results = self.model(frame)[0]
                frame = results.plot()
                out.write(frame)
                self.update_ui_signal.emit(cur_num, total)
            else:
                break
        # 释放资源
        cap.release()
        out.release()
    def stop(self):
        self.is_running = False
if __name__ == "__main__":
    app = QApplication(sys.argv)
    win = MainWindow()
    win.show()
    sys.exit(app.exec_())
```